

EXPERIMENT-2

SACHIN SINGH

24UADS1045

Objective :-

WAP to implement a multi-layer perceptron (MLP) network with one hidden layer using numpy in Python. Demonstrate that it can learn the XOR Boolean function.

Description Of the Model :-

Multi-Layer Perceptron (MLP) for XOR using Sigmoid Activation

Model Type :-

The model is a feedforward Artificial Neural Network (ANN) consisting of :

1. Input Layer
2. Hidden Layer
3. Output Layer

It is trained using the Backpropagation Algorithm with Gradient Descent.

Architecture :-

1. Input Layer
2. Number of neurons: 2
3. Inputs: x_1, x_2
4. Represents XOR input pairs:
(0,0), (0,1), (1,0), (1,1)

Hidden Layer :-

1. Number of neurons: 4
2. Activation Function: Sigmoid
3. Purpose:
 - Introduces non-linearity
 - Transforms inputs into a higher-dimensional space
 - Enables solving non-linearly separable problems like XOR

Output Layer :-

1. Number of neurons: 1
2. Activation Function: Sigmoid
3. Produces output between 0 and 1
4. Thresholded at 0.5 for binary classification

Activation Function :-

The model uses Sigmoid Function:

$$\sigma(z) = 1 / (1 + e^{-z})$$

Non-linear

- Differentiable
- Required for backpropagation
- Suitable for binary classification

Forward Propagation

The output is computed in two steps:

Hidden Layer:

$$Z1 = XW1 + b1$$

$$A1 = \sigma(Z1)$$

Output Layer:

$$Z2 = A1W2 + b2$$

$$Y^{\wedge} = \sigma(Z2)$$

Loss Function :-

The model uses Mean Squared Error (MSE):

$$\text{Loss} = (1/n) \sum (Y - Y^{\wedge})^2$$

It measures the difference between actual and predicted outputs.

Backpropagation :-

Error is propagated backward to update weights:

1. Compute output error
2. Compute hidden layer error
3. Update weights using gradient descent:

$$W = W + \eta \cdot \text{gradient}$$

Where:

$$\eta = \text{Learning rate}$$

Training Parameters :-

1. Learning Rate: Efficiently chosen (e.g., 0.5)
2. Maximum Epochs: 2000
3. Early Stopping when accuracy = 100%
4. Random weight initialization

Hidden Layer Importance :-

1. XOR is not linearly separable.
2. A single perceptron cannot solve XOR because:
3. It cannot create a non-linear decision boundary.

The hidden layer:

1. Maps inputs to a higher dimension
2. Creates non-linear separation

3. Enables correct classification

Final Performance :-

1. Achieves 100% accuracy
2. Correctly classifies:
 - 0, 0 $\rightarrow$ 0
 - 0, 1 $\rightarrow$ 1
 - 1, 0 $\rightarrow$ 1
 - 1, 1 $\rightarrow$ 0

Source Code :-

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import time

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def sigmoid_derivative(x):
    return x * (1 - x)

# XOR Input
X = np.array([[0,0],
              [0,1],
              [1,0],
              [1,1]])

# XOR Output
y = np.array([[0],
              [1],
              [1],
              [0]])

np.random.seed(42)

input_neurons = 2
```

```
hidden_neurons = 4
```

```
output_neurons = 1
```

```
learning_rate = 0.5
```

```
epochs = 2000
```

```
# Initialize weights and biases
```

```
W1 = np.random.uniform(-1, 1, (input_neurons, hidden_neurons))
```

```
b1 = np.random.uniform(-1, 1, (1, hidden_neurons))
```

```
W2 = np.random.uniform(-1, 1, (hidden_neurons, output_neurons))
```

```
b2 = np.random.uniform(-1, 1, (1, output_neurons))
```

```
loss_history = []
```

```
accuracy_history = []
```

```
weight_history = []
```

```
for epoch in range(epochs):
```

```
    # ---- Forward Pass ----
```

```
    hidden_input = np.dot(X, W1) + b1
```

```
    hidden_output = sigmoid(hidden_input)
```

```
    final_input = np.dot(hidden_output, W2) + b2
```

```
    predicted_output = sigmoid(final_input)
```

```
    # ---- Loss ----
```

```
    loss = np.mean((y - predicted_output) ** 2)
```

```
    loss_history.append(loss)
```

```
    # ---- Accuracy ----
```

```

predictions_binary = (predicted_output > 0.5).astype(int)
accuracy = np.mean(predictions_binary == y)
accuracy_history.append(accuracy)

# ---- Backpropagation ----
output_error = y - predicted_output
output_delta = output_error * sigmoid_derivative(predicted_output)

hidden_error = output_delta.dot(W2.T)
hidden_delta = hidden_error * sigmoid_derivative(hidden_output)

# ---- Update Weights ----
W2 += hidden_output.T.dot(output_delta) * learning_rate
b2 += np.sum(output_delta, axis=0, keepdims=True) * learning_rate

W1 += X.T.dot(hidden_delta) * learning_rate
b1 += np.sum(hidden_delta, axis=0, keepdims=True) * learning_rate

weight_history.append(W1.copy())

# ---- Display Table ----
df = pd.DataFrame({
    "Input1": X[:,0],
    "Input2": X[:,1],
    "Target": y.flatten(),
    "Predicted": predicted_output.flatten(),
})

```

```

print(f"\nEpoch {epoch+1}")

print(df)

print(f"Loss: {loss:.6f} | Accuracy: {accuracy*100:.2f}%")

# ---- Early Stopping ----

if accuracy == 1:
    print("\nConvergence Achieved!")
    break

time.sleep(0.3)

print("\nFinal Weights and Biases")

print("\nW1:\n", W1)
print("\nb1:\n", b1)

print("\nW2:\n", W2)
print("\nb2:\n", b2)

print(f"\nFinal Accuracy: {accuracy*100:.2f}%")

# 1 Loss vs Epoch

plt.figure()

plt.plot(loss_history)

plt.title("Loss vs Epochs")

plt.xlabel("Epoch")

plt.ylabel("Loss")

plt.show

```

2 Accuracy vs Epoch

```
plt.figure()
plt.plot(accuracy_history)
plt.title("Accuracy vs Epochs")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.show()
```

3 Weight Evolution (First Hidden Layer)

```
weight_array = np.array(weight_history)

plt.figure()
plt.plot(weight_array[:,0,0])
plt.title("Weight W1[0,0] Evolution")
plt.xlabel("Epoch")
plt.ylabel("Weight Value")
plt.show()
```

4 Decision Surface

```
xx, yy = np.meshgrid(np.linspace(-0.5,1.5,100),
                     np.linspace(-0.5,1.5,100))
```

```
grid = np.c_[xx.ravel(), yy.ravel()]
hidden = sigmoid(np.dot(grid, W1) + b1)
output = sigmoid(np.dot(hidden, W2) + b2)
output = output.reshape(xx.shape)
```

```
plt.figure()
plt.contourf(xx, yy, output)
```

```
plt.scatter(X[:,0], X[:,1], c=y.flatten())  
plt.title("Decision Boundary")  
plt.show()
```

```
# Decision Surface with Better Colors
```

```
xx, yy = np.meshgrid(np.linspace(-0.5,1.5,200),  
                     np.linspace(-0.5,1.5,200))
```

```
grid = np.c_[xx.ravel(), yy.ravel()]
```

```
hidden = sigmoid(np.dot(grid, W1) + b1)  
output = sigmoid(np.dot(hidden, W2) + b2)  
output = output.reshape(xx.shape)
```

```
plt.figure()
```

```
# Clear two-color separation
```

```
plt.contourf(xx, yy, output > 0.5, alpha=0.6, cmap='coolwarm')
```

```
# Plot actual data points
```

```
plt.scatter(X[:,0], X[:,1],  
           c=y.flatten(),  
           cmap='bwr',  
           edgecolors='black',  
           s=150)
```

```
plt.title("Decision Boundary (XOR)")
```

```
plt.xlabel("Input 1")
```

```
plt.ylabel("Input 2")  
plt.colorbar(label="Class Region")  
plt.show()
```

Output :-

Epoch 1

	Input1	Input2	Target	Predicted
0	0	0	0	0.268179
1	0	1	1	0.267541
2	1	0	1	0.224191
3	1	1	0	0.226489

Loss: 0.315398 | Accuracy: 50.00%

Epoch 2

	Input1	Input2	Target	Predicted
0	0	0	0	0.312461
1	0	1	1	0.310000
2	1	0	1	0.267059
3	1	1	0	0.267931

Loss: 0.295680 | Accuracy: 50.00%

Epoch 3

	Input1	Input2	Target	Predicted
0	0	0	0	0.356456
1	0	1	1	0.352075
2	1	0	1	0.310798
3	1	1	0	0.309991

Loss: 0.279491 | Accuracy: 50.00%

Epoch 4

	Input1	Input2	Target	Predicted
--	--------	--------	--------	-----------

0	0	0	0	0.396344
---	---	---	---	----------

1	0	1	1	0.390195
---	---	---	---	----------

2	1	0	1	0.351377
---	---	---	---	----------

3	1	1	0	0.348883
---	---	---	---	----------

Loss: 0.267846 | Accuracy: 50.00%

Epoch 5

	Input1	Input2	Target	Predicted
--	--------	--------	--------	-----------

0	0	0	0	0.429595
---	---	---	---	----------

1	0	1	1	0.421996
---	---	---	---	----------

2	1	0	1	0.385843
---	---	---	---	----------

3	1	1	0	0.381862
---	---	---	---	----------

Loss: 0.260412 | Accuracy: 50.00%

.

.

.

Epoch 314

	Input1	Input2	Target	Predicted
--	--------	--------	--------	-----------

0	0	0	0	0.500007
---	---	---	---	----------

1	0	1	1	0.557957
---	---	---	---	----------

2	1	0	1	0.454772
---	---	---	---	----------

3	1	1	0	0.461042
---	---	---	---	----------

Loss: 0.238811 | Accuracy: 50.00%

Epoch 315

	Input1	Input2	Target	Predicted
--	--------	--------	--------	-----------

0	0	0	0	0.499819
---	---	---	---	----------

1	0	1	1	0.558417
---	---	---	---	----------

2	1	0	1	0.454508
---	---	---	---	----------

3	1	1	0	0.460820
---	---	---	---	----------

Loss: 0.238683 | Accuracy: 75.00%

Epoch 316

	Input1	Input2	Target	Predicted
0	0	0	0	0.499628
1	0	1	1	0.558881
2	1	0	1	0.454242
3	1	1	0	0.460597

Loss: 0.238554 | Accuracy: 75.00%

Epoch 317

	Input1	Input2	Target	Predicted
0	0	0	0	0.499435
1	0	1	1	0.559349
2	1	0	1	0.453976
3	1	1	0	0.460371

Loss: 0.238423 | Accuracy: 75.00%

.
.
.

Epoch 526

	Input1	Input2	Target	Predicted
0	0	0	0	0.411279
1	0	1	1	0.699204
2	1	0	1	0.495959
3	1	1	0	0.355058

Loss: 0.159938 | Accuracy: 75.00%

Epoch 527

	Input1	Input2	Target	Predicted
0	0	0	0	0.410392
1	0	1	1	0.699818

2 1 0 1 0.497569

3 1 1 0 0.354143

Loss: 0.159096 | Accuracy: 75.00%

Epoch 528

Input1 Input2 Target Predicted

0 0 0 0 0.409490

1 0 1 1 0.700435

2 1 0 1 0.499200

3 1 1 0 0.353221

Loss: 0.158247 | Accuracy: 75.00%

Epoch 529

Input1 Input2 Target Predicted

0 0 0 0 0.408573

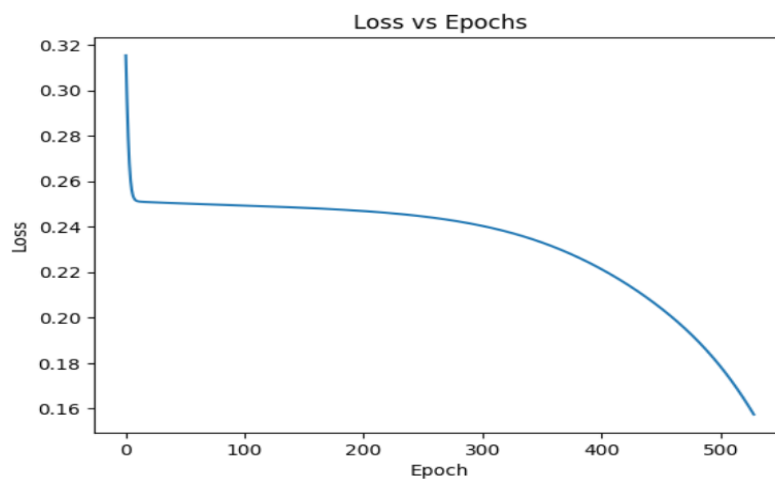
1 0 1 1 0.701054

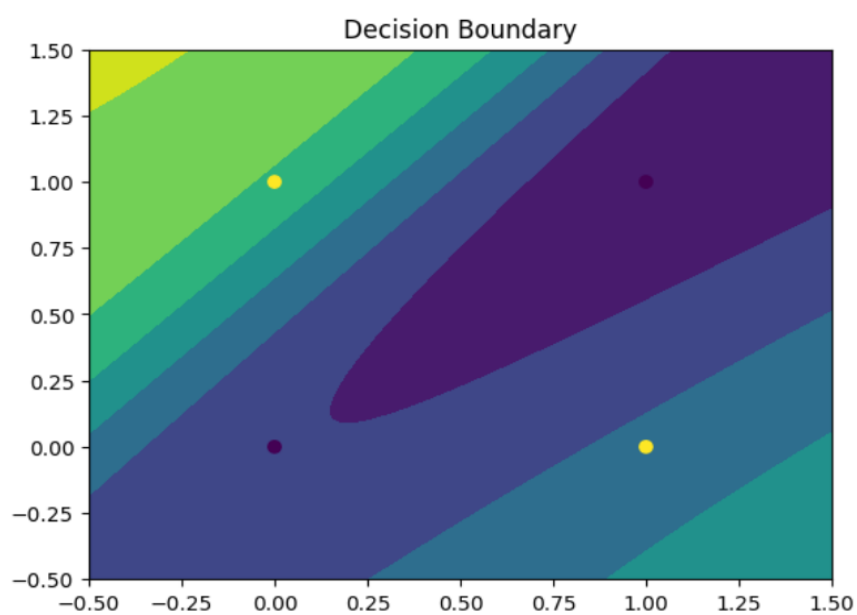
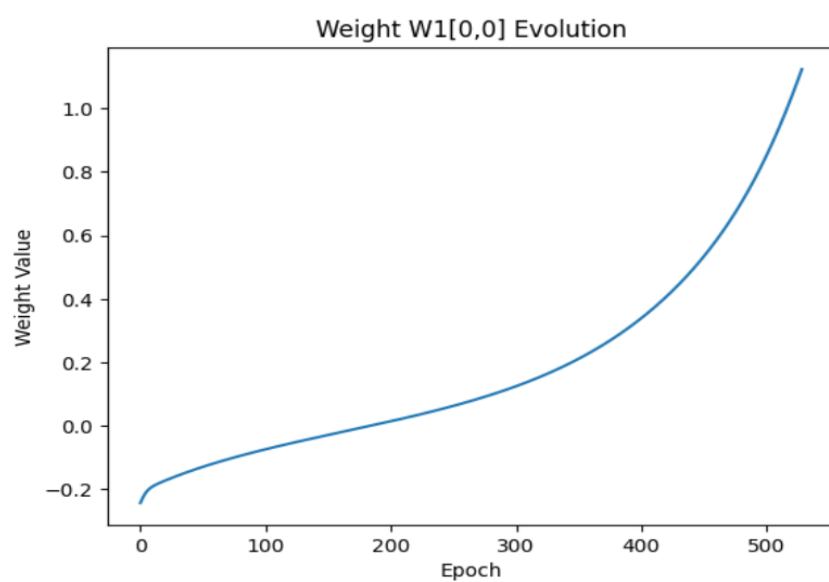
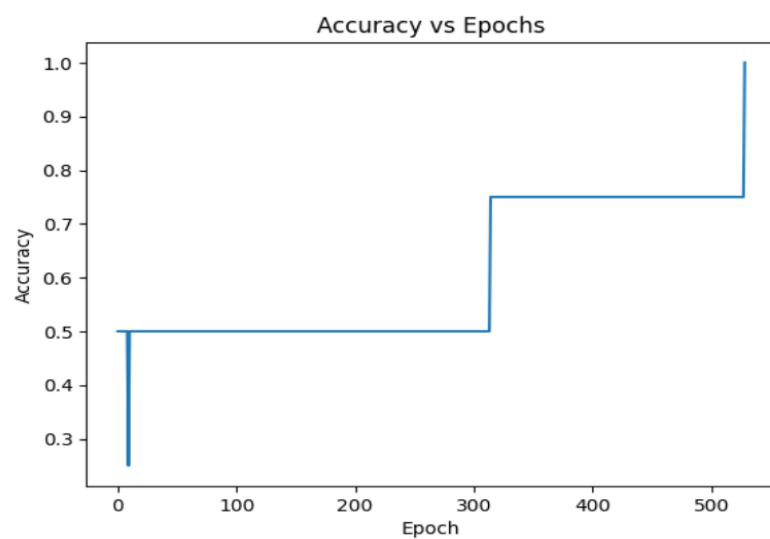
2 1 0 1 0.500853

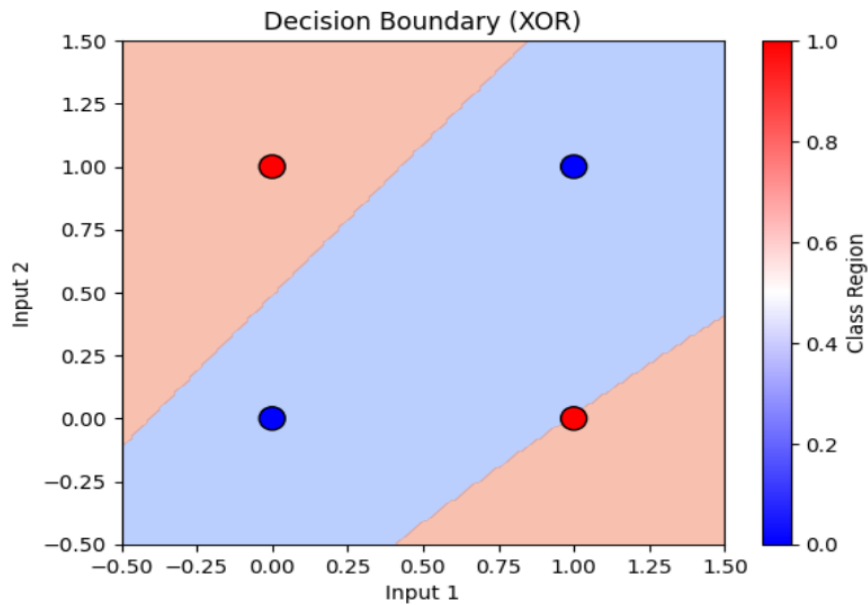
3 1 1 0 0.352292

Loss: 0.157390 | Accuracy: 100.00%

Convergence Achieved!







Description of Code :-

Importing Libraries :-

1. numpy → Matrix operations (weights, dot products)
2. pandas → Display epoch output as table
3. matplotlib → Plot graphs
4. time → Add 300 ms delay after each epoch

Defining Activation Function :-

1. sigmoid() → Introduces non-linearity
2. sigmoid_derivative() → Required for backpropagation

Defining XOR Dataset :-

1. X → Input features
2. y → Target output

Represents XOR truth table

Initializing Network Parameters :-

1. W1 → Input to hidden layer weights
2. W2 → Hidden to output weights
3. b1, b2 → Bias terms

Random initialization prevents symmetry problem

Forward Propagation :-

1. Compute hidden layer output
2. Compute final prediction
3. Apply sigmoid activation at each layer

Loss Calculation :-

1. Calculates Mean Squared Error (MSE)
2. Measures difference between actual and predicted output

Backpropagation :-

1. Compute output layer error
2. Propagate error backward
3. Calculate gradients for weight updates

Updating Weights :-

1. Adjust weights using gradient descent
2. Learning rate controls update step size

Epoch Output Table :-

Displays:

1. Inputs
2. Target values
3. Predicted outputs

Helps observe learning progress

Delay :-

1. Adds 300 milliseconds pause
2. Makes training visually observable

Graph Plotting :-

Four graphs are plotted:

1. Loss vs Epochs
2. Accuracy vs Epochs
3. Weight Evolution
4. Decision Boundary

Purpose:

1. Visualize convergence
2. Observe training behavior
3. Show non-linear separation for XOR

Early Stopping :-

1. Stops training when model achieves 100% accuracy
2. Prevents unnecessary epochs
3. Avoids overfitting

Interpretation of Results :-

The implemented Multi-Layer Perceptron successfully learned the XOR Boolean function using:

1. Sigmoid activation
2. Backpropagation
3. Gradient descent

The results show that the MLP successfully learned the non-linearly separable XOR function. Loss decreased steadily, accuracy reached 100%, and the decision boundary confirmed non-linear separation. This validates the effectiveness of hidden layers in neural networks.

Performance Evaluation :-

The performance of the Multi-Layer Perceptron (MLP) model is evaluated using classification accuracy, defined as the ratio of correctly classified input patterns to the total number of patterns.

For the XOR gate, the MLP achieves 100% accuracy after a certain number of epochs. The accuracy gradually increases during training and stabilizes at 1.0, indicating successful convergence. This improvement occurs because the hidden layer enables the network to create a non-linear decision boundary, allowing it to correctly classify the XOR function.

The loss decreases steadily across epochs, confirming effective learning through backpropagation and gradient descent. Unlike a single-layer perceptron, the MLP does not suffer from convergence limitations for XOR, as it can model non-linearly separable patterns.

Increasing the number of epochs beyond convergence does not significantly improve accuracy, since the model has already reached the optimal solution. Early stopping ensures efficient training and prevents unnecessary weight updates.

My Comments :-

The things I learned in this experiment are as follows :-

1. Single layer perceptron is a simple step function but here we have used activation function like here I used sigmoid as we can differentiate it from step function and in class I got to know that we use it because the loss function is easy to calculate using gradient descent and also sigmoid is differentiable and continuous.
2. Using the multilayer perceptron we can solve the XOR problem that was in SLP. As SLP can only separate linear decision boundaries and MLP can also solve non-linear decision boundaries. I learned how a single hidden layer can change the model's capabilities, as I learned about forward propagation and back propagation updated the weights.
3. The learning and no of epochs is different from previous experiment here the algorithm took more than 500 epochs to get an accuracy of 100% where as in SLP the accuracy was getting 0% after few epochs. The learning took more time but was able to differentiate non-linear decision boundaries.
4. By graph of loss vs epoch I saw how initially the loss decrease significantly and after few epochs the loss was not changing much till 300 epochs and after that loss was decreasing exponentially and after the 529th epoch the loss was lowest and the convergence achieved there.
5. By graph of accuracy vs epoch I learned in the beginning the accuracy was 50% but at 315 epoch the accuracy changed to 75 % and at 529 to 100%.
6. By graph of decision boundary I saw how the non-linear decision boundary of XOR is differentiated using different shades of different colours.

